Spring Boot — First Spring Boot Application

Interview Revision + Quick Recall Notes

> **Source:** Lecture 2 — Writing Our First Spring Boot Application
> **Goal:** Understand the flow, remember the important concepts, and explain them clearly in interviews.

▼ 1. What Are We Building?

We build a simple web application where:

Browser → sends HTTP request → Spring Boot application → sends response → Browser

Example endpoint:

```
http://localhost:8080/hello
```

Response:

```
Hello World
```

Questions hidden behind this simple example

- How does the browser communicate with Java?
- How does Java understand a URL?
- Who starts the server?
- Who listens on port `8080`?
- How does `/hello` get connected to a Java method?

⭐ **Interview Important:** Spring Boot hides much of this complexity initially, but understanding the underlying flow is important for debugging and interviews.

▼ 2. IP Address vs Port Number

IP Address

An **IP address identifies the machine** on a network.

Example:

```
192.168.1.10
```

Port

A **port number identifies the application/service running on that machine**.

Example:

```
192.168.1.10:8080
```

Meaning:

```
Machine      → 192.168.1.10
Application  → listening on port 8080
```

Easy Recall

```
IP Address → Which machine?
Port       → Which application on that machine?
```

Diagram

```
                  Network
                     |
                     v
              192.168.1.10
          +----------------------+
          | Chrome       :5000   |
          | WhatsApp     :5222   |
          | Zoom         :8801   |
          | Spring Boot  :8080   |
          +----------------------+
```

🔥 **Must Remember:** The IP address brings the data to the correct machine; the port helps the operating system deliver it to the correct application.

▼ 3. Localhost

`localhost` means **your own machine**.

It is equivalent to:

```
localhost = 127.0.0.1
```

Therefore:

```
localhost:8080
```

means:

> Send the request to my own machine on port `8080`.

If the Spring Boot application is running on port `8080`:

```
http://localhost:8080
```

can be used to access it.

Quick Recall

```
localhost → my own machine
127.0.0.1 → loopback address for my own machine
:8080     → port 8080
```

⭐ **Interview Important:** `localhost` identifies the local machine; the port identifies the application/service listening there.

▼ 4. Default HTTP and HTTPS Ports

Browsers normally use default ports, so we usually do not write them explicitly.

Protocol	Default Port
HTTP	8
HTTPS	44

Examples:

```
http://www.example.com
        ↓
      port 80
```

```
https://www.example.com
        ↓
      port 443
```

For a local Spring Boot application using port `8080`, we normally specify it:

```
http://localhost:8080
```

DNS + Port

For:

```
https://www.example.com
```

Conceptually:

```
Domain name
     |
     v
    DNS
     |
     v
  IP Address
     |
     +---- HTTPS → default port 443
```

🔥 **Must Remember:**

- DNS resolves a domain name to an IP address.
- The port is either explicitly written or determined by the protocol's default.

▼ 5. Development Setup

The lecture uses **IntelliJ IDEA** for Spring Boot development.

It provides support for:

- Java
- Maven
- Spring Boot
- Code navigation
- Auto-completion
- Project structure
- Debugging

Check Java installation:

```
java -version
```

⭐ **Interview/Practical Point:** Before creating a Spring Boot project, make sure Java is installed and working.

▼ 6. Spring Initializr

What problem does it solve?

Creating a Spring Boot project manually involves repetitive setup:

- Folder structure
- Maven configuration
- Spring Boot version
- Java version
- Dependencies
- Package structure
- Project metadata

What is Spring Initializr?

Spring Initializr is a project generator for Spring applications.

It creates the basic Spring Boot project skeleton so developers can start writing application logic quickly.

It allows us to choose

- Project type
- Programming language
- Spring Boot version
- Project metadata
- Packaging type
- Java version
- Dependencies

🔥 **Must Remember:**

> Spring Initializr = quickly generates the basic structure/configuration of a Spring Boot project.

▼ 7. Dependency

Definition

A **dependency is an external library required by our project**.

Example:

If our application needs to connect to MySQL, it needs the MySQL connector library.

Examples of dependencies:

- Spring Web
- MySQL Connector
- Spring Data JPA
- Spring Security
- Lombok
- Hibernate

In Maven projects, dependencies are declared in:

```
pom.xml
```

Quick Recall

```
Project
   |
   +---- needs external library
               |
               v
          Dependency
```

⭐ **Interview Important:** A library becomes a dependency when our project requires it to perform its work.

▼ 8. Spring Boot Version Types

SNAPSHOT

A **SNAPSHOT** is a work-in-progress version.

- May contain bugs
- Not considered a final release

RC — Release Candidate

A **Release Candidate** is almost ready for final release but is not yet the final stable release.

Stable Release

A stable release is tested and suitable for normal development.

🔥 **Must Remember:** For normal development and real projects, prefer a stable release unless there is a specific reason to use another version.

▼ 9. JAR Packaging

JAR stands for:

Java Archive

Modern Spring Boot applications commonly use **JAR packaging**.

A Spring Boot JAR can contain:

- Application code
- Required setup to run the application

This makes applications easy to run directly rather than requiring manual deployment to an external server.

⭐ **Recall:**

```
Spring Boot Application
          |
          v
        JAR
          |
          v
     Run Application
```

▼ 10. Important Spring Boot Project Structure

Common important files/folders:

```
project
├── src/main/java
├── src/main/resources
├── pom.xml
└── application.properties
```

`src/main/java`

Contains Java source code.

`src/main/resources`

Contains configuration files and static resources.

`pom.xml`

Maven configuration file containing:

- Project information
- Dependencies

`application.properties`

Used to configure the Spring Boot application.

Example:

```
server.port=9090
```

Main Application Class

The class from which the Spring Boot application starts.

Example:

```
@SpringBootApplication
public class FirstSpringBootApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            FirstSpringBootApplication.class,
            args
        );
    }
}
```

⭐ **Must Remember:** `main()` starts the Java application, and `SpringApplication.run(...)` starts/configures the Spring Boot application.

▼ 11. Controller

What is a Controller?

A **Controller is the entry point for incoming web requests.**

Simple analogy:

> Controller = receptionist of the application.

It receives a request and determines what response should be returned.

Flow

```
Browser
   |
   | Request
   v
Controller
   |
   | Response
   v
Browser
```

⭐ **Interview Important:** In a typical Spring MVC application, the controller handles incoming web requests and connects the request to application logic.

▼ 12. @RestController

`@RestController` is a Spring annotation.

It tells Spring that the class can handle HTTP requests and return data directly as a response.

Example:

```
@RestController
public class HelloController {
}
```

🔥 **Must Remember:**

```
@RestController
       ↓
Class can handle web requests
```

▼ 13. Creating the First Endpoint

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World";
    }
}
```

Understand each part

Code	Meaning
`@RestController`	Class can handle HTTP requests
`@GetMapping("/hello")`	Maps a GET request to `/hello`
`sayHello()`	Java method executed
`return "Hello World"`	Response returned

Request Flow

```
GET /hello
    |
    v
Spring MVC
    |
    | Finds mapping
    v
sayHello()
    |
    | returns
    v
"Hello World"
    |
    v
Browser
```

When we open:

```
http://localhost:8080/hello
```

The browser displays:

```
Hello World
```

🔥 **Must Remember:** `@GetMapping("/hello")` connects the GET request for `/hello` to the `sayHello()` method.

▼ 14. Embedded Tomcat

When the application starts, we may see:

```
Tomcat started on port 8080
```

Important point:

We did **not**:

- Install Tomcat separately
- Write socket programming code
- Manually create a server

Spring Boot includes and configures an **embedded Tomcat server** when the web dependency is used.

Correct Mental Model

```
Spring Boot
    |
    | starts/configures
    v
Embedded Tomcat
    |
    | listens on port
    v
HTTP Requests
```

Responsibility

```
Spring Boot → starts/configures the application
Tomcat      → listens for HTTP requests
Spring MVC  → maps request to controller method
```

🔥 **Must Remember:**

> Tomcat is the actual server listening on the configured port.

▼ 15. Changing the Server Port

Default port:

```
8080
```

Change it in `application.properties`:

```
server.port=9090
```

After restarting:

```
http://localhost:9090
```

Endpoint:

```
http://localhost:9090/hello
```

Important Flow

```
application.properties
        |
        | server.port=9090
        v
Spring Boot configuration
        |
        v
Embedded Tomcat
        |
        v
Listens on port 9090
```

🔥 **Must Remember:** Spring Boot reads the configuration, while the embedded Tomcat server listens on the configured port.

▼ 16. Complete Request → Response Flow ⭐

This is one of the most important diagrams to remember.

Complete Flow

```
Browser
   |
   | GET /hello
   v
localhost:8080
   |
   v
Embedded Tomcat
   |
   v
Spring MVC
   |
   | Find matching mapping
   v
@GetMapping("/hello")
   |
   v
sayHello()
   |
   | return "Hello World"
   v
HTTP Response
   |
   v
Browser
   |
   v
Hello World
```

Step-by-step

1. Browser sends a request to:

   ```
   localhost:8080/hello
   ```

2. Request reaches the embedded Tomcat server.
3. Spring MVC checks available mappings.

4. `/hello` is matched with `sayHello()`.
5. `sayHello()` returns `"Hello World"`.
6. Response goes back to the browser.
7. Browser displays `Hello World`.

🔥 **Interview Answer:**

> The browser sends an HTTP request to the configured port. Embedded Tomcat receives the request, Spring MVC finds the matching controller mapping, invokes the controller method, and the returned data is sent back as the HTTP response.

▼ 17. What Spring Boot Does Behind the Scenes

Although our code is very small:

```
@RestController
public class HelloController {

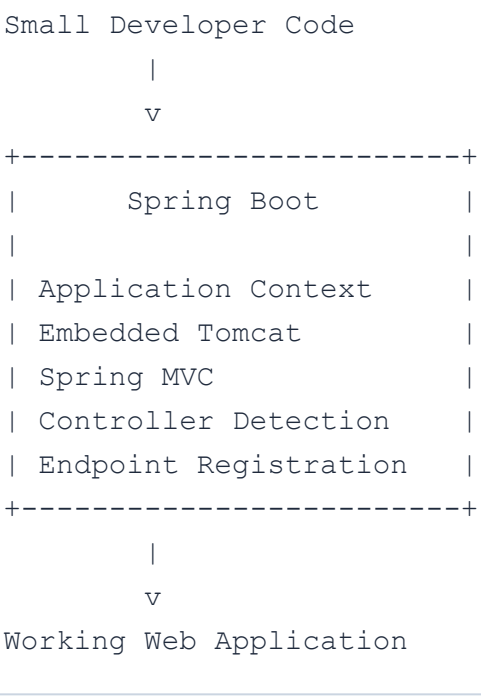
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World";
    }
}
```

Spring Boot performs many tasks behind the scenes.

Important tasks

- Creates the application context
- Starts embedded Tomcat
- Configures Spring MVC
- Detects the controller class
- Registers `/hello`
- Handles HTTP request/response
- Returns data to the browser

Mental Model



⭐ **Interview Important:** Spring Boot reduces repetitive infrastructure setup so developers can focus more on business logic.

▼ 18. Why Learn Spring Boot Internals?

Spring Boot makes development fast, but internal understanding is necessary when something goes wrong.

Knowing only annotations is not enough.

For example, if the endpoint returns:

```
404 Not Found
```

We should be able to investigate:

- Is the controller detected?
- Is the package structure correct?
- Did component scanning happen?
- Was the bean created?
- Did Spring MVC register the mapping?
- Is the URL correct?
- Did Tomcat receive the request?

Another example: Slow Startup

If the application takes 10 seconds to start, possible causes include:

- Too many beans being created
- Database connection taking time
- Auto-configuration doing extra work
- Heavy startup logic
- External service calls blocking startup

Controller Not Receiving Request

Possible reasons:

- Wrong port
- Wrong URL
- Application not running
- Controller not detected
- Incorrect annotation

- Context path configured
- Request blocked by security

🔥 **Must Remember:**

> Spring Boot gives speed; understanding Spring fundamentals and internals gives control.

▼ 19. Industry Perspective

Basic code such as:

```
@RestController
@GetMapping("/hello")
```

is easy to write and can be generated by modern tools.

The valuable engineering skills are understanding:

- Why something is not working
- How different layers communicate
- How to debug production issues
- How to design maintainable systems
- How to make applications reliable

⭐ **Interview Perspective:** Do not just memorize annotations. Be able to explain the flow behind them.

🔥 Final Quick Revision

Core Definitions

Concept	One-line Recall
IP Address	Identifies the machine
Port	Identifies the application/service on that machine
Localhost	Refers to the local machine
`127.0.0.1`	Loopback address for the local machine
Spring Initializr	Generates a Spring Boot project skeleton
Dependency	External library required by the project
JAR	Java Archive; common packaging for Spring Boot applications
Controller	Entry point for incoming web requests
`@RestController`	Marks a class as capable of handling web requests
`@GetMapping`	Maps a GET request to a method
Tomcat	Embedded server that listens for HTTP requests
Spring MVC	Handles request mapping to controller methods

Important Numbers

```
HTTP  → 80
HTTPS → 443
Spring Boot default → 8080
```

Most Important Diagram

```
Browser
   |
   | HTTP Request
   v
localhost:8080/hello
   |
   v
Embedded Tomcat
   |
   v
Spring MVC
   |
   v
@GetMapping("/hello")
   |
   v
sayHello()
   |
   v
"Hello World"
   |
   | HTTP Response
   v
Browser
```

⭐ Interview Questions

1. What is a port?

A port helps identify the application/service running on a particular machine.

2. What is the difference between IP and port?

```
IP   → Which machine?
Port → Which application on that machine?
```

3. What is localhost?

`localhost` refers to the local machine and is commonly associated with `127.0.0.1`.

4. Why don't we specify the port for normal HTTP/HTTPS URLs?

Because HTTP and HTTPS have default ports:

```
HTTP  → 80
HTTPS → 443
```

5. What is Spring Initializr?

A project generator that creates the basic structure and configuration for a Spring Boot application.

6. What is a dependency?

An external library required by the project.

7. What does `@RestController` do?

It marks a class as capable of handling HTTP requests and returning data directly as a response.

8. What does `@GetMapping("/hello")` do?

It maps a GET request for `/hello` to the annotated Java method.

9. Who listens on port 8080?

The embedded Tomcat server listens on the configured port.

10. What happens when `/hello` is requested?

```
Browser
 → Tomcat
 → Spring MVC
 → matching controller method
 → response
 → Browser
```

11. How do you change the Spring Boot port?

In `application.properties`:

```
server.port=9090
```

12. Why should we learn Spring Boot internals?

Because internal understanding helps diagnose problems such as 404 errors, startup delays, controller detection issues, incorrect mappings, and request failures.

🔥 Must Remember

1. **IP → machine, Port → application/service.**
2. **localhost → local machine.**
3. **localhost:8080 → local machine + port 8080.**
4. **HTTP default port = 80.**
5. **HTTPS default port = 443.**
6. **Spring Initializr generates the project skeleton.**
7. **Dependencies are external libraries required by the project.**
8. `pom.xml` **contains Maven project information and dependencies.**
9. `application.properties` **is used for Spring Boot configuration.**
10. `@RestController` **marks a class as a web controller.**
11. `@GetMapping("/hello")` **maps a GET request to a Java method.**
12. **Embedded Tomcat listens for HTTP requests.**
13. **Spring MVC maps requests to controller methods.**
14. **Spring Boot hides infrastructure complexity, but understanding the internals is essential for debugging.**
15. **Spring Boot gives speed; Spring fundamentals give control.**

🧠 30-Second Recall

```
Browser
   ↓
URL = localhost:8080/hello
   ↓
Tomcat receives request
   ↓
Spring MVC finds /hello mapping
   ↓
Controller method executes
   ↓
"Hello World"
   ↓
HTTP Response
   ↓
Browser
```

Understand → Recall → Revise → Explain